

“小龙虾”项目二次开发实战

理论介绍

- 1 OpenClaw 产品形态与研发价值
- 2 AI 编程下的研发心法

经验总结

- 10 Vibe Coding 编程在
项目二开领域的
经验分享

实战演练

3 本地部署与
MVP 验证

4 完整源码
拆解流程

5 接入自定义
Agent 模型类

6 接入飞书
工作流实战

7 接入自定义
Agent Skills

8 前端 UI 定制化
开发与优化

9 项目维护 Spec Docs 规范生成

✧ 全部围绕 OpenClaw 项目展开

从入门到深度定制

部署验证

源码拆解

深度定制

学习收获

Vibe Coding 能力进阶

5 项能力

01 开源项目快速选型能力

- 根据业务需求筛选开源项目
- 判断项目成熟度与可扩展性
- 避免盲目跟风或无效调研

核心能力：
从“找工具”升级为“做技术选型”

02 开源项目快速落地能力

- 快速本地部署与环境搭建
- 理解项目运行搭建
- 建立可复用启动流程

实际价值：
让开源项目真正可用，不只是 Sar

03 AI 辅助源码理解能力

- 借助 AI 理解复杂项目
- 拆解架构与模板关系
- 快速建立系统认知

核心认知：
AI 是理解复杂系统的加速器

04 企业级二次开发能力

- 基于现有项目扩展个性化需求
- 接入业务系统与企业平台
- 实现可持续迭代的定制化能力

本质：从“用工具”升级为“改造工具”

05 长期可迭代的 VibeCoding 开发链路

- AI 协作开发流程
- Spec 驱动研发习惯
- 可持续维护的工程体系

最终目标

形成“指哪打哪”的 AI 开发能力

课程总结

这节课不是教一个项目

而是教你如何驾驭 开源 + AI 完成真实项目落地



基于开源项目做二次开发

—— 项目产品落地的最高效路径，没有之一

核心优势

产品形态已验证

成然交互设计 + 用户验证的产品形态 → 降低产品试错成本



技术基础已封装

登录/权限/UI框架现成可用 → 减少重复造轮子，专注业务创新



社区生态支持

Issues/PR/插件可复用 → 问题快速解决 (Agent/RAG/Workflow尤明显)



技术趋势同步

新模型/新工具接入快 + 社区持续更新 → 保持技术前沿能力



现实挑战

技术栈适配风险

技术框架可能非团队/个人擅长技术领域



架构可控性下降

原作者设计思想未知 + 修改核心模块成本高 → 潜在架构锁定风险



定制需求不匹配

冗余功能多/必需功能缺 + Fork维护压力大 → 定制复杂度提升



长期维护不确定

项目可能停止维护 + License变化/安全漏洞 → 企业必须评估风险



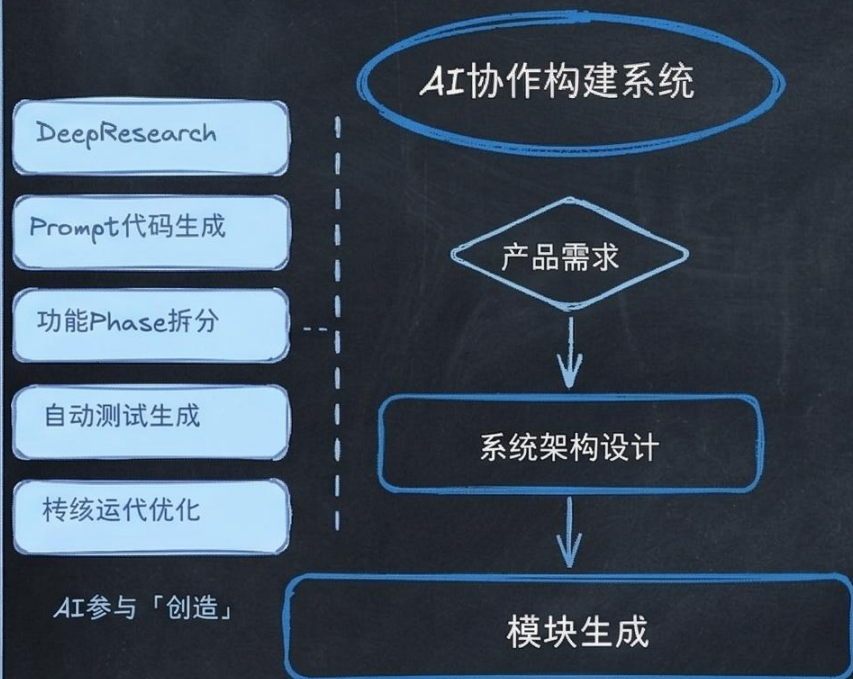
VS

AI 时代的工程趋势

自研决定上限 | 开源决定起步速度 | 二次开发能力 ≠ 抄代码，而是工程整合能力

Vibe Coding 项目开发心智模型

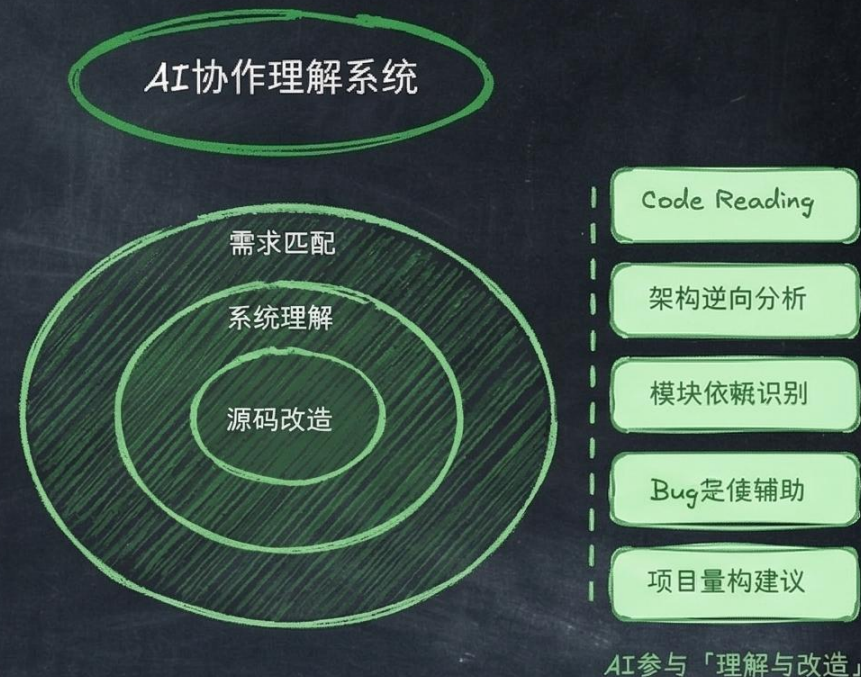
从零开发 Build-Oriented



先设计“世界”，再写代码

从 0 → 1 创造全新系统
架构先行 · Prompt驱动 · 模块化生成

二次开发 Understand-Oriented



先理解“世界”，再修改代码

从 1 → N 扩展改造已有系统
理解优先 · 逆向分析 · 渐进式修改

Vibe Coding 二次开发可复用的链路

核心原则：先跑 → 再验证 → 后理解 → 再设计 → 最后改造

“不是我理解了再改，而是我验证过，才值得我理解”



工具链支撑

Git

Cursor

Codex

LLM

飞书

linear

Skills

Vibe Coding 核心实践

- * 先确保能跑起来，再谈理解和改造
- * MVP 验证是关键：证明价值后才值得深入
- * Spec 驱动设计：从二开视角重新规划 Phase
- * 工程收敛：保持代码整洁，移除非必要文件
- * 持续交付：版本管理 + 部署上线

关键思维转变

传统思维：理解 → 设计 → 开发 → 验证

Vibe 思维：运行 → 验证 → 理解 → 设计 → 改造

核心：验证过才值得理解，能跑的代码优于完美的设计



01 正确的工程顺序

clone → 看源码 → 看懵 → 放弃

✓ 先跑通 → 再理解 → 再定制

二次开发最正确的工程顺序

02 Cursor 角色定位

不是「写代码」「改代码」工具

启动

二开

MVP验证

定制开发

项目瘦身

UI定制

Cursor 是「意图执行器」

与 Vibe Coding 完全一致

03 Spec Docs (phasex.md)

理解系统



重构目标

而不是「边看边改」

区分「会二开」和
「乱改源码」的分水岭

01 最小兼容原则

任性地考虑自己的架构

✓ 最小程度兼容原有系统

二开 ≠ 重构，是「嫁接」

02 先跑通，再规范

完成二开



规范化项目

二开过程中改动非常多，没必要同步维护 Docs

不要急着做系统架构设计

完成后再规范化

03 让 AI 帮你拆解庞大代码

顶层分析



逐步深入

代码庞大？让大模型帮你梳理结构

先顶层分析，再逐步分析

做到清晰可见的架构